

CodingLab4

May 17, 2026

Neural Data Science

Lecturer: Prof. Dr. Philipp Berens, Dr. Jan Lause

Tutors: Jonas Beck, Kyra Kadhim, Jonathan Oesterle, Julius Würzler

Summer term 2026

Student names: Zhidong Zhang, Yuzhe Han, Bach Nguyen

LLM Disclaimer: Github Copilot was used to help with function calling and structure prompting, Claude and Gemini were used (by different members) to help with function understanding and outcome analysis.

1 Coding Lab 4

In this notebook you will work with electrophysiological recordings, that have already been converted into spike counts for a population of cells from the Macaque V1. During the experiment the animal has been presented with several drifting grating stimuli, in response to which the neural activity was recorded. In this exercise sheet we will study how you can visualize the activity of multiple neural spike trains and assess whether a neuron is selective to a specific stimulus type.

Download the data files `nds_cl_4_*.csv` from ILIAS and save it in the subfolder `../data/`. We recommend you to use a subset of the data for testing and debugging, ideally focus on a single cell (e.g. cell number x). The spike times and stimulus conditions are read in as pandas data frames. You can solve the exercise by making heavy use of that, allowing for many quite compact computations. See [documentation](#) and several good [tutorials](#) on how to do this. Of course, converting the data into classical numpy arrays is also valid.

```
[10]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib as mpl
import numpy as np
import scipy.optimize as opt

from scipy import signal as signal
from typing import Tuple

import itertools
```

```
%matplotlib inline

%load_ext jupyter_black

%load_ext watermark
%watermark --time --date --timezone --updated --python --iversions --watermark
↳-p sklearn
```

The jupyter_black extension is already loaded. To reload it, use:

```
%reload_ext jupyter_black
```

The watermark extension is already loaded. To reload it, use:

```
%reload_ext watermark
```

Last updated: 2026-05-17 19:21:21 CEST

Python implementation: CPython

Python version : 3.10.0

IPython version : 8.39.0

sklearn: 1.7.2

matplotlib: 3.10.8

numpy : 2.2.6

pandas : 2.3.3

scipy : 1.15.3

seaborn : 0.13.2

Watermark: 2.6.0

```
[11]: plt.style.use("../matplotlib_style.txt")
```

1.1 Load data

```
[12]: spikes = pd.read_csv("../data/nds_cl_4_spiketimes.csv") # neuron id, spike time
      stims = pd.read_csv("../data/nds_cl_4_stimulus.csv") # stimulus onset in ms,
      ↳direction

      stimDur = 2000.0 # in ms
      nTrials = 11 # number of trials per condition
      nDirs = 16 # number of conditions
      deltaDir = 22.5 # difference between conditions

      stims["StimOffset"] = stims["StimOnset"] + stimDur
```

We require some more information about the spikes for the plots and analyses we intend to make later. With a solution based on dataframes, it is natural to compute this information here and add it as additional columns to the `spikes` dataframe by combining it with the `stims` dataframe. We later need to know which condition (`Dir`) and trial (`Trial`) a spike was recorded in, the relative

spike times compared to stimulus onset of the stimulus it was recorded in (`relTime`) and whether a spike was during the stimulation period (`stimPeriod`). But there are many options how to solve this exercise and you are free to choose any of them.

```
[13]: # you may add computations as specified above
spikes["Dir"] = np.nan
spikes["relTime"] = np.nan
spikes["Trial"] = np.nan
spikes["stimPeriod"] = np.nan # add new columns

dirs = np.unique(stims["Dir"])
trialcounter = np.zeros_like(dirs)

for i, row in stims.iterrows():
    trialcounter[dirs == row["Dir"]] += 1

    i0 = spikes["SpikeTimes"] > row["StimOnset"]
    i1 = spikes["SpikeTimes"] < row["StimOffset"]

    select = i0.values & i1.values # mask of those spike within the stimulus
    ↪duration

    spikes.loc[select, "Dir"] = row["Dir"]
    spikes.loc[select, "Trial"] = trialcounter[dirs == row["Dir"]][0]
    spikes.loc[select, "relTime"] = spikes.loc[select, "SpikeTimes"] -
    ↪row["StimOnset"]
    spikes.loc[select, "stimPeriod"] = True

spikes = spikes.dropna()

# print("Number of neurons: ", len(np.unique(spikes["Neuron"])))
```

```
/var/folders/ty/jw9csq0j64j3_w_b8kmz3b2h0000gn/T/ipykernel_12074/2352568024.py:2
1: FutureWarning: Setting an item of incompatible dtype is deprecated and will
raise an error in a future version of pandas. Value 'True' has dtype
incompatible with float64, please explicitly cast to a compatible dtype first.
    spikes.loc[select, "stimPeriod"] = True
```

```
[14]: spikes.head()
```

```
[14]:
```

	Neuron	SpikeTimes	Dir	relTime	Trial	stimPeriod
514	1	15739.000000	270.0	169.000000	1.0	True
515	1	15776.566667	270.0	206.566667	1.0	True
516	1	15808.466667	270.0	238.466667	1.0	True
517	1	15821.900000	270.0	251.900000	1.0	True
518	1	15842.966667	270.0	272.966667	1.0	True

1.2 Task 1: Plot spike rasters

In a raster plot, each spike is shown by a small tick at the time it occurs relative to stimulus onset. Implement a function `plotRaster()` that plots the spikes of one cell as one trial per row, sorted by conditions (similar to what you saw in the lecture). Why are there no spikes in some conditions and many in others?

If you opt for a solution without a dataframe, you need to change the interface of the function.

Grading: 3 pts

```
[15]: def plotRaster(spikes: pd.DataFrame, neuron: int, savePath=None):
        """plot spike rasters for a single neuron sorted by condition

        Parameters
        -----

        spikes: pd.DataFrame
            Pandas DataFrame with columns
            Neuron | SpikeTimes | Dir | relTime | Trial | stimPeriod

        neuron: int
            Neuron ID

        Note
        ----

        this function does not return anything, it just creates a plot!
        """

        # -----
        # Write a raster plot function for the data (2 pts)
        # -----
        data = spikes.loc[spikes["Neuron"] == neuron, ["Dir", "Trial", "relTime"]]

        # divide into two parts
        mask_left = data["Dir"] < 180
        data_panels = [data[mask_left], data[~mask_left]]

        fig, axes = plt.subplots(1, 2, figsize=(15, 6), dpi=300, sharex=True)

        for ax, data_subset in zip(axes, data_panels):
            assert not data_subset.empty, "The subset of data is empty!"

            sorted_data = data_subset.sort_values(["Dir", "Trial", "relTime"]).
            ↪reset_index(
                drop=True
            ) # sorting
```

```

grouped_spikes = (
    sorted_data.groupby(["Dir", "Trial"], sort=False)["relTime"]
    .apply(list) # each group, turn it into list of relTime
    .reset_index() # turn ["Dir", "Trial"] from group index to normal
↳ columns
) # group

spike_sequences = grouped_spikes["relTime"].values

# raster plot
ax.eventplot(
    spike_sequences,
    orientation="horizontal",
    linelengths=0.75,
    colors="black",
    linewidths=0.5,
)

dir_boundaries = [] # boundaries
dir_centers = [] # where to show labels
dir_labels = []

current_idx = 0
for direction, reltimes in grouped_spikes.groupby("Dir", sort=False):
    n_trials = len(reltimes)
    start_idx = current_idx
    end_idx = current_idx + n_trials

    dir_boundaries.append(end_idx - 0.5)
    dir_centers.append(start_idx + (n_trials - 1) / 2.0)
    dir_labels.append(f"Dir {direction}")

    current_idx = end_idx

for boundary in dir_boundaries[:-1]:
    ax.axhline(boundary, color="gray", linestyle="--", alpha=0.5,
↳ linewidth=0.8)

ax.set_yticks(dir_centers)
ax.set_yticklabels(dir_labels)
ax.set_ylim(-0.5, len(grouped_spikes) - 0.5)

ax.set_xlabel("Relative Time (ms)", fontweight="bold")

axes[0].set_ylabel("Direction of motion (degree)", fontweight="bold")

```

```

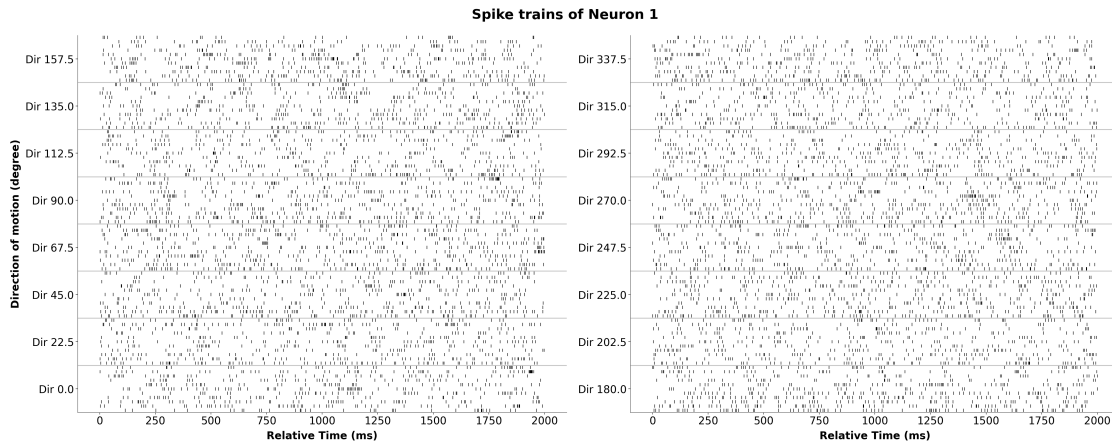
plt.suptitle(f"Spike trains of Neuron {neuron}", fontweight="bold",
fontsize=13)
plt.tight_layout()

if savePath:
    plt.savefig(savePath)
plt.show()

plotRaster(spikes, 1)

```

/var/folders/ty/jw9csq0j64j3_w_b8kmz3b2h0000gn/T/ipykernel_12074/1824787070.py:8
3: UserWarning: The figure layout has changed to tight
plt.tight_layout()



Find examples of 1. a direction selective neuron 2. an orientation selective neuron 3. neither and explain your choices.

1. A direction selective neuron: 13, as it shows clearly more firing for 180 degrees (and 157.5 degrees) than others.
2. An orientation selective neuron: 21, as it shows clearly more firing for 67.5 and 90 degrees as well as 247.5 and 270 degrees.
3. Neither: 1, as it doesn't show clear difference of firing across different degrees.

```

[16]: # -----
# Find and explain examples? (1 pt)
# -----

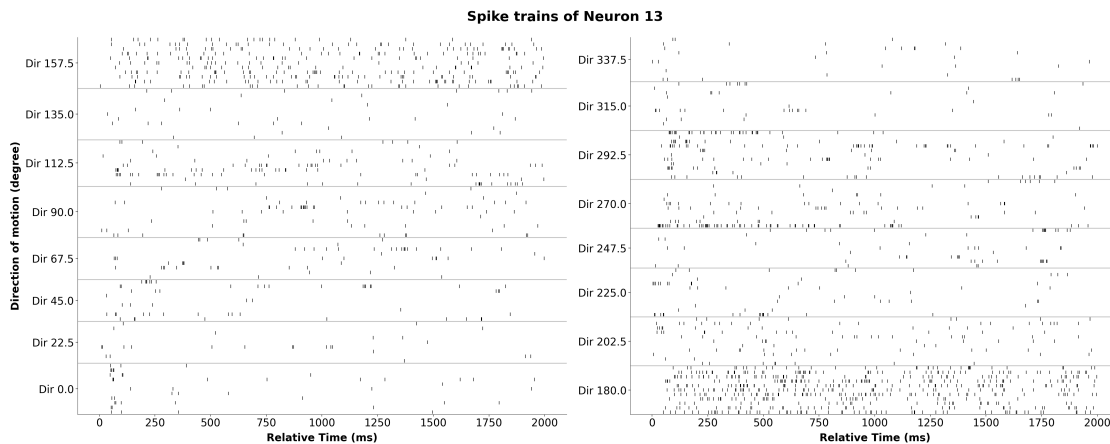
# for neuron in np.unique(spikes["Neuron"]):
#     plotRaster(

```

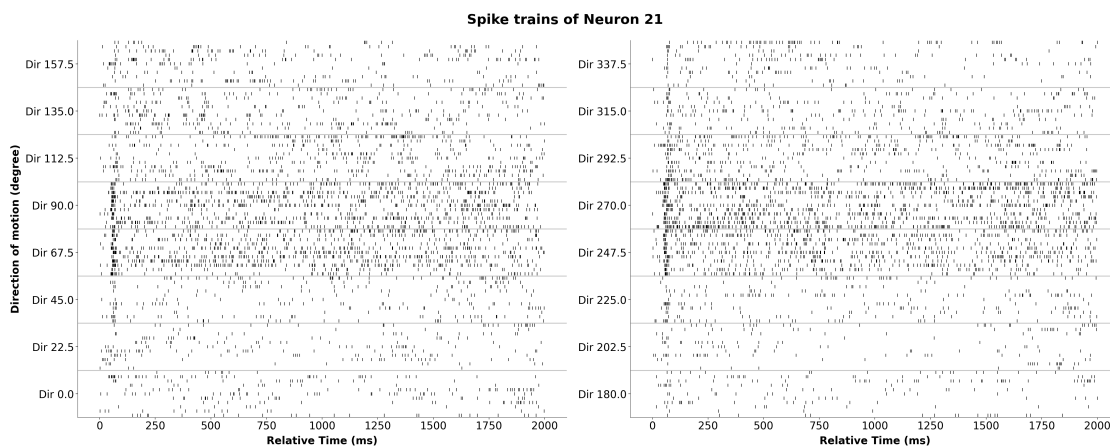
```
#         spikes, neuron, savePath=f"../outputs/
↳Lab4_spike_trains_neuron_{neuron}.jpg"
#     )

# Direction selective
plotRaster(spikes, 13)
# Orientation selective
plotRaster(spikes, 21)
# neither selective
plotRaster(spikes, 1)
```

/var/folders/ty/jw9csq0j64j3_w_b8kmz3b2h0000gn/T/ipykernel_12074/1824787070.py:8
 3: UserWarning: The figure layout has changed to tight
 plt.tight_layout()



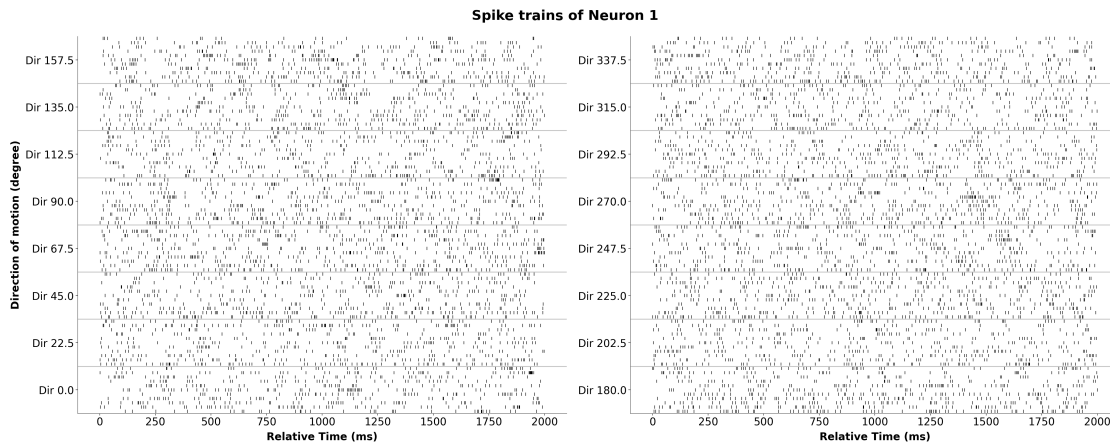
/var/folders/ty/jw9csq0j64j3_w_b8kmz3b2h0000gn/T/ipykernel_12074/1824787070.py:8
 3: UserWarning: The figure layout has changed to tight
 plt.tight_layout()



/var/folders/ty/jw9csq0j64j3_w_b8kmz3b2h0000gn/T/ipykernel_12074/1824787070.py:8

3: UserWarning: The figure layout has changed to tight

plt.tight_layout()



1.3 Task 2: Plot spike density functions

Compute an estimate of the spike rate against time relative to stimulus onset. There are two ways:

* Discretize time: Decide on a bin size, count the spikes in each bin and average across trials. *
Directly estimate the probability of spiking using a density estimator with specified kernel width.

For full points, the optimal kernel- or bin-width needs to be computed.

Implement one of them in the function `plotPSTH()`. If you don't use a dataframe you may need to change the interface of the function.

Grading: 4 pts

```
[17]: def select_bin_size(spikes: pd.DataFrame) -> int:
      """Compute the optimal bin size for a discretized bar-PSTH

      Parameters
      -----

      spikes: pd.DataFrame
          Pandas DataFrame with columns
          Dir | relTime | Trial

      Return
      -----

      bin_size: int
          the optimal bin size computed
      """
```



```

n_bin_list = np.arange(10, 10000, 1)
loss_list = []

all_spikes = spikes.groupby(["Dir"])["relTime"].apply(lambda x: x.values)
all_trials_num = spikes.groupby(["Dir"])["Trial"].apply(lambda x: len(x.
↪unique()))

for n_bin in n_bin_list:
    loss = 0.0 # the loss over all Dir
    bin_size = stimDur / n_bin
    bins = np.linspace(0, stimDur, n_bin + 1)

    for spikes_dir, trials_num in zip(all_spikes, all_trials_num):
        spikes_dir = np.asarray(spikes_dir)
        n_spikes, _ = np.histogram(spikes_dir, bins=bins)
        avg = n_spikes.mean()
        var = n_spikes.var()
        loss += (2 * avg - var) / (trials_num * bin_size) ** 2

    loss_list.append(loss)

opt_bin = stimDur / n_bin_list[np.argmin(loss_list)]
return opt_bin

def plotPSTH(spikes: pd.DataFrame, neuron: int):
    """Plot PSTH for a single neuron sorted by condition

    Parameters
    -----

    spikes: pd.DataFrame
        Pandas DataFrame with columns
            Neuron | SpikeTimes | Dir | relTime | Trial | stimPeriod

    neuron: int
        Neuron ID

    Note
    ----

    this function does not return anything, it just creates a plot!
    """

    # -----

```

```

# Implement one of the spike rate estimates (3 pts)
# -----
neuron_spikes = spikes[spikes["Neuron"] == neuron]
bin_size = select_bin_size(neuron_spikes[["Dir", "relTime", "Trial"]])

bins = np.arange(0, stimDur + bin_size, bin_size)
num_trials = len(neuron_spikes["Trial"].unique())

fig, axes = plt.subplots(8, 2, figsize=(15, 8), sharex=True, sharey=True)
axes = axes.flatten()

for idx, dir in enumerate(dirs):
    # -----
    # Plot the obtained spike rate estimates (0.5 pts)
    # -----
    if idx < 8:
        ax = axes[14 - 2 * idx]
    else:
        ax = axes[15 - 2 * idx]
    dir_spikes = neuron_spikes[neuron_spikes["Dir"] == dir]["relTime"]

    # Calculate rates in Hz: (counts per bin) / (num_trials *
    ↪ bin_size_in_seconds)
    weights = np.ones_like(dir_spikes) / (num_trials * (bin_size / 1000.0))

    ax.hist(dir_spikes, bins=bins, weights=weights, color="black")
    ax.set_title(f"Dir {dir}°")
    ax.set_xlim(0, stimDur)

    if idx == 0 or idx == 8:
        ax.set_xlabel("Relative Time (ms)", fontweight="bold")
    if idx == 3:
        ax.set_ylabel("Spike Rate (Hz)", fontweight="bold")
    plt.suptitle(
        f"PSTH for Neuron {neuron} (bin size: {bin_size:.2f} ms)",
        fontweight="bold",
        fontsize=16,
    )
plt.tight_layout()
plt.show()

```

Plot the same 3 examples you selected in Task 1

How did you choose your bandwidths and why? (0.5 pts)

We implemented the algorithm by ([Shimazaki et al. 2007](#)), which finds the binwidths

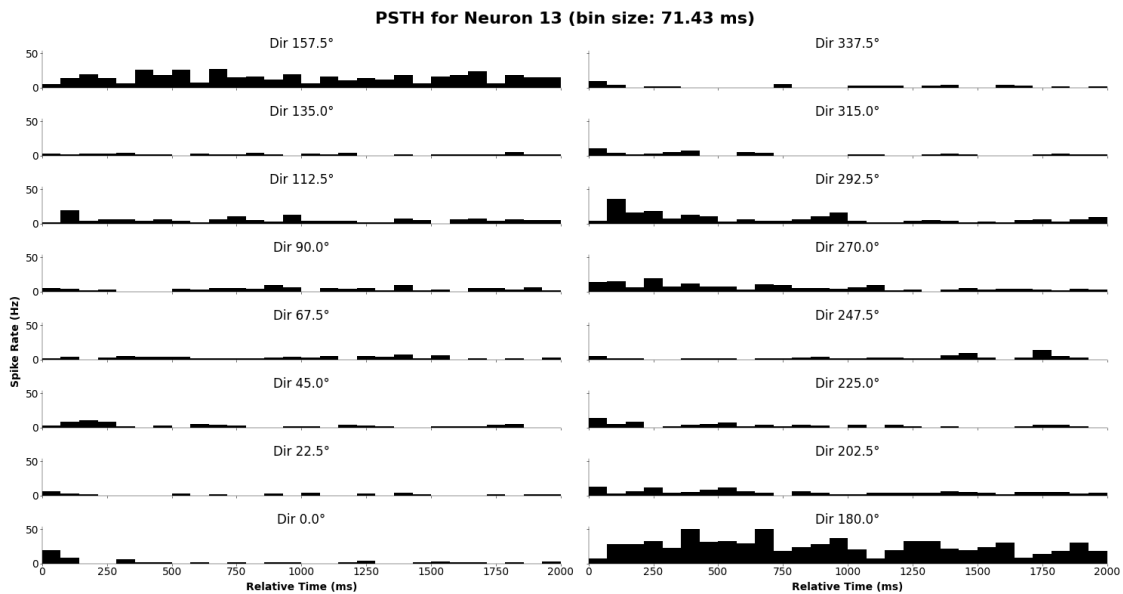
that minimize the loss function

$$C_n(\Delta) = \frac{2\bar{k} - v}{(n\Delta)^2}$$

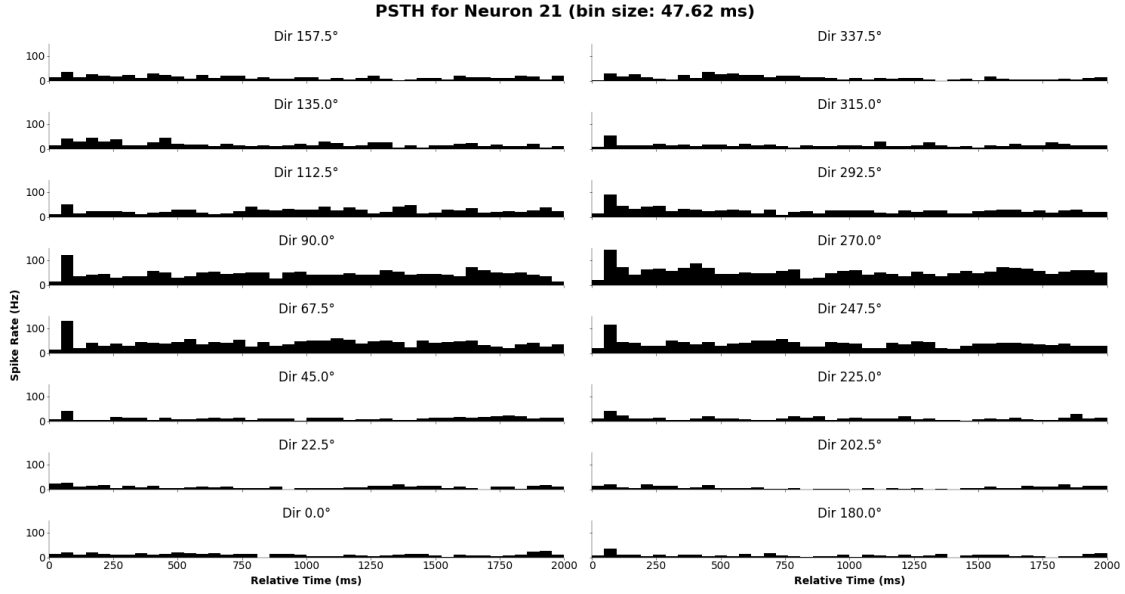
Note that we denoted the loss over a neuron as the summation of loss over all stimulus ('Dir') of the neuron.

```
[18]: plotPSTH(spikes, 13)
      plotPSTH(spikes, 21)
      plotPSTH(spikes, 1)
```

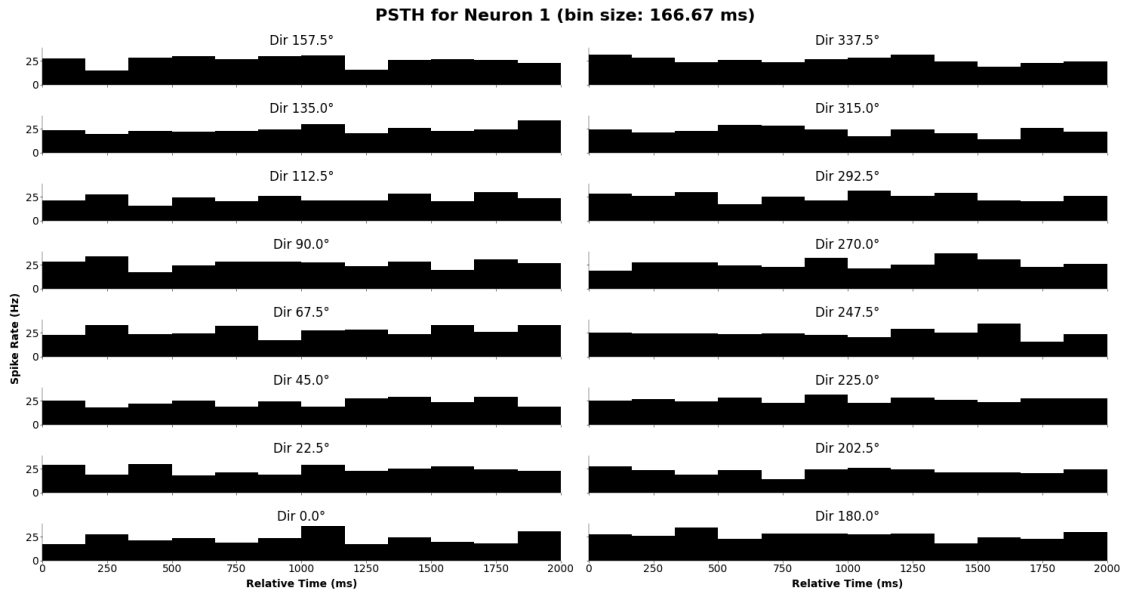
```
/var/folders/ty/jw9csq0j64j3_w_b8kmz3b2h0000gn/T/ipykernel_12074/3016376343.py:9
9: UserWarning: The figure layout has changed to tight
  plt.tight_layout()
```



```
/var/folders/ty/jw9csq0j64j3_w_b8kmz3b2h0000gn/T/ipykernel_12074/3016376343.py:9
9: UserWarning: The figure layout has changed to tight
  plt.tight_layout()
```



```
/var/folders/ty/jw9csq0j64j3_w_b8kmz3b2h0000gn/T/ipykernel_12074/3016376343.py:9
9: UserWarning: The figure layout has changed to tight
plt.tight_layout()
```



1.4 Task 3: Fit and plot tuning functions

The goal is to visualize the activity of each neuron as a function of stimulus direction. First, compute the spike counts of each neuron for each direction of motion and trial. The result should

be a matrix $\mathbf{x}$, where x_{jk} represents the spike count of the j -th response to the k -th direction of motion (i.e. each column contains the spike counts for all trials with one direction of motion). If you used dataframes above, the `groupby()` function allows to implement this very compactly. Make sure you don't lose trials with zero spikes though. Again, other implementations are completely fine.

Fit the tuning curve, i.e. the average spike count per direction, using a von Mises model. To capture the non-linearity and direction selectivity of the neurons, we will fit a modified von Mises function:

$$f(\theta) = \exp(\alpha + \kappa(\cos(2 * (\theta - \phi)) - 1) + \nu(\cos(\theta - \phi) - 1))$$

Here, θ is the stimulus direction. Implement the von Mises function in `vonMises()` and plot it to understand how to interpret its parameters ϕ , κ , ν , α . Perform a non-linear least squares fit using a package/function of your choice. Implement the fitting in `tuningCurve()`.

Plot the average number of spikes per direction, the spike counts from individual trials as well as your optimal fit.

Select two cells that show nice tuning to test your code.

Grading: 5 pts

```
[19]: def vonMises( : np.ndarray, : float, : float, : float, : float) -> np.ndarray:
    """Evaluate the parametric von Mises tuning curve with parameters p at
    ↪ locations theta.

    Parameters
    -----

    : np.array, shape=(N, )
        Locations. The input unit is degree.

    , , , : float
        Function parameters

    Return
    -----
    f: np.array, shape=(N, )
        Tuning curve.
    """

    # -----
    # Implement the Mises model (0.5 pts)
    # -----
    theta = np.deg2rad( ) # in radian
    phi = np.deg2rad( ) # in radian

    f = np.exp( + * (np.cos(2 * (theta - phi)) - 1) + * (np.cos(theta -
    ↪ phi) - 1))
```

```
f[f < 0] = 0 # firing rate cannot be negative

return f
```

```
[20]: # -----
# plot von Mises curves with varying parameters and explain what they do (2 pts)
# -----
thetas = np.linspace(0, 360, 360)

# Default parameters
alpha_default, kappa_default, nu_default, phi_default = 1.0, 1.0, 1.0, 180.0

# Parameter variations
alphas = [0.0, 1.0, 2.0]
kappas = [0.0, 1.0, 3.0]
nus = [0.0, 1.0, 3.0]
phis = [90.0, 180.0, 270.0]

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()

# alpha
for a in alphas:
    y = vonMises(thetas, a, kappa_default, nu_default, phi_default)
    axes[0].plot(thetas, y, label=rf"$\alpha={a}$")
axes[0].set_title(r"Varying offset ($\alpha$)")
axes[0].legend()
axes[0].set_xlabel("Direction (°)")
axes[0].set_ylabel("Firing rate")

# phi
for p in phis:
    y = vonMises(thetas, alpha_default, kappa_default, nu_default, p)
    axes[1].plot(thetas, y, label=rf"$\phi={p}^\circ$")
axes[1].set_title(r"Varying preferred direction ($\phi$)")
axes[1].legend()
axes[1].set_xlabel("Direction (°)")

# kappa
for k in kappas:
    y = vonMises(thetas, alpha_default, k, nu_default, phi_default)
    axes[2].plot(thetas, y, label=rf"$\kappa={k}$")
axes[2].set_title(r"Varying orientation tuning width ($\kappa$)")
axes[2].legend()
axes[2].set_xlabel("Direction (°)")
axes[2].set_ylabel("Firing rate")
```

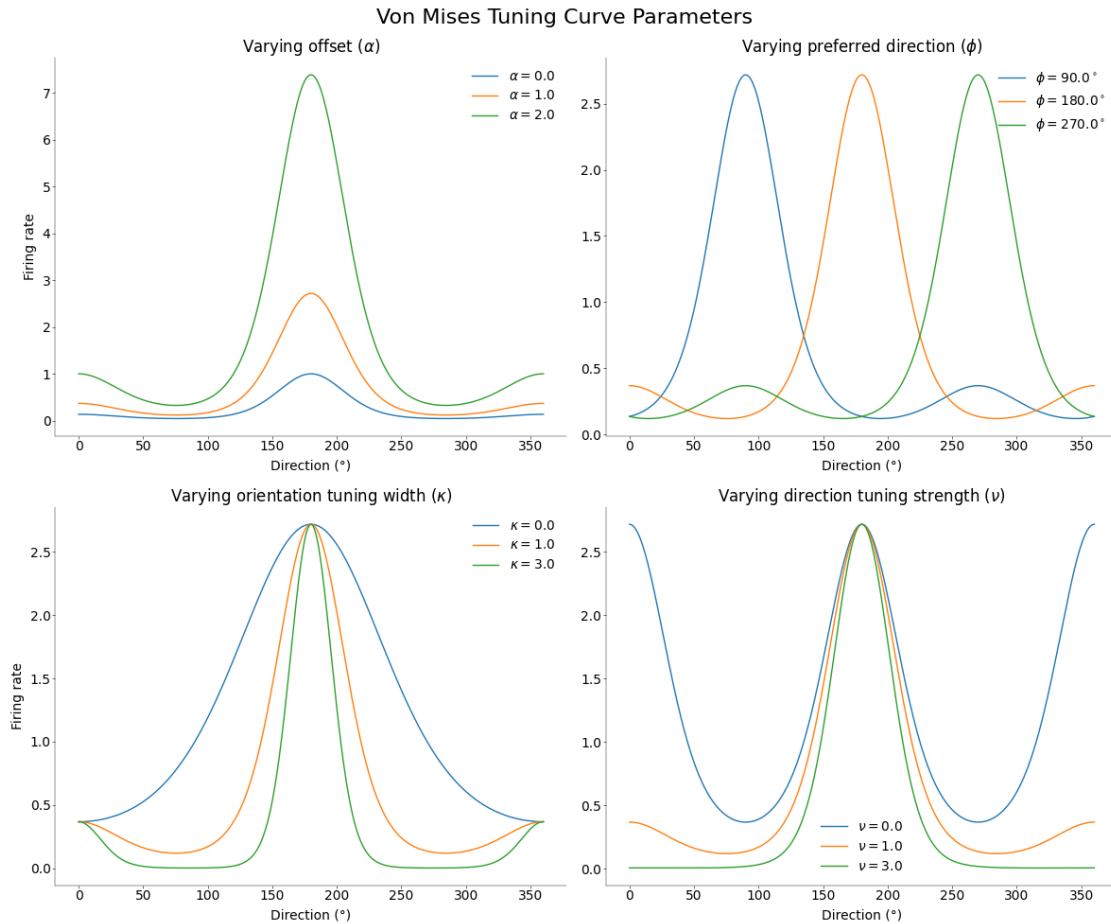
```

# nu
for n in nus:
    y = vonMises(thetas, alpha_default, kappa_default, n, phi_default)
    axes[3].plot(thetas, y, label=r"$\nu$={n}")
axes[3].set_title(r"Varying direction tuning strength ($\nu$)")
axes[3].legend()
axes[3].set_xlabel("Direction (°)")

plt.suptitle("Von Mises Tuning Curve Parameters", fontsize=16)
plt.tight_layout()
plt.show()

```

/var/folders/ty/jw9csq0j64j3_w_b8kmz3b2h0000gn/T/ipykernel_12074/520264175.py:54
: UserWarning: The figure layout has changed to tight
plt.tight_layout()



- α determines the baseline firing level regardless of the tuning properties.
- ϕ is the preferred orientation that induces the highest firing rate.

- κ determines the strength of orientation tuning; increasing it sharpens the tuning curve around the preferred orientation.
- ν determines the strength of direction tuning; increasing it suppresses responses to non-preferred directions, making the tuning curve less flat.

```
[21]: def residuals(params, theta, y_observed):
    return vonMises(theta, *params) - y_observed

def tuningCurve(counts: np.ndarray, dirs: np.ndarray, show: bool = True) -> np.
    ndarray:
    """Fit a von Mises tuning curve to the spike counts in count with direction_
    dir using a least-squares fit.

    Parameters
    -----

    counts: np.array, shape=(total_n_trials, )
        the spike count during the stimulation period

    dirs: np.array, shape=(total_n_trials, )
        the stimulus direction in degrees

    show: bool, default=True
        Plot or not.

    Return
    -----
    p: np.array or list, (4,)
        parameter vector of tuning curve function
    """

    # -----
    # Compute the spike count matrix (0.5 pts)
    # -----
    unique_dirs = np.unique(dirs)
    x = np.zeros(shape=(nTrials, nDirs))
    for k, dir in enumerate(unique_dirs):
        x[:, k] = counts[dirs == dir]

    # -----
    # fit the von Mises tuning curve to the spike counts (0.5 pts)
    # -----
    mean_count = np.mean(x, axis=0)
    # true_rate = np.mean(x, axis=0) / stimDur * 1000
    p0 = [
```



```

    np.log(np.max(mean_count) + 1e-6),
    1.0,
    1.0,
    unique_dirs[np.argmax(mean_count)],
] # initial params

# 1. curve_fit
p, cov = opt.curve_fit(vonMises, unique_dirs, mean_count, p0=p0)

# 2. least_squares
# res = opt.least_squares(residuals, x0=p0, args=(unique_dirs, mean_count))
# p = res.x

if show:
    # -----
    # plot the data and fitted tuning curve (1 pt)
    # -----
    plt.figure(figsize=(8, 5))

    # Plot individual trials (gray dots)
    for k, d in enumerate(unique_dirs):
        counts_trials = x[:, k]
        # rate_trials = x[:, k] / stimDur * 1000

        plt.scatter(
            np.full(nTrials, d),
            counts_trials,
            color="gray",
            alpha=0.6,
            label="Individual trials" if k == 0 else None,
        )
        plt.errorbar(
            d,
            np.mean(counts_trials),
            yerr=np.std(counts_trials),
            fmt="o",
            color="black",
            markersize=8,
            capsize=5,
            elinewidth=1,
            markeredgecolor="white",
            # label="Mean ± SD" if k == 0 else None,
        )

    # Plot average spike count (black dots)
    plt.plot(unique_dirs, mean_count, "ko", label="Average spike count")

```

```

    # Plot fitted tuning curve (red line)
    thetas = np.linspace(0, 360, 360)
    fitted_curve = vonMises(thetas, *p)
    plt.plot(thetas, fitted_curve, "r-", linewidth=2, label="von Mises fit")

    plt.title("Direction Tuning Curve")
    plt.xlabel("Stimulus direction (°)")
    plt.ylabel("Spike counts")
    # plt.ylabel("Firing rate (spikes / sec)")
    plt.legend()
    plt.show()

    return p

```

Plot tuning curve and fit for different neurons. Good candidates to check are 28, 29 or 37.

```

[22]: def get_data(spikes, neuron):
    spk_by_dir = (
        spikes[spikes["Neuron"] == neuron]
        .groupby(["Dir", "Trial"])["stimPeriod"]
        .sum()
        .astype(int)
        .reset_index()
    )

    dirs = spk_by_dir["Dir"].values
    counts = spk_by_dir["stimPeriod"].values

    # because we count spikes only when they are present, some zero entries in
    # the count vector are missing
    for i, Dir in enumerate(np.unique(spikes["Dir"])):
        m = nTrials - np.sum(dirs == Dir)
        if m > 0:
            dirs = np.concatenate((dirs, np.ones(m) * Dir))
            counts = np.concatenate((counts, np.zeros(m)))

    idx = np.argsort(dirs)
    dirs_sorted = dirs[idx] # sorted dirs
    counts_sorted = counts[idx]

    return dirs_sorted, counts_sorted

```

```

[23]: # -----
# plot the average number of spikes per direction, the spike
# counts from individual trials as well as your optimal fit
# for different neurons (0.5 pts)
# -----

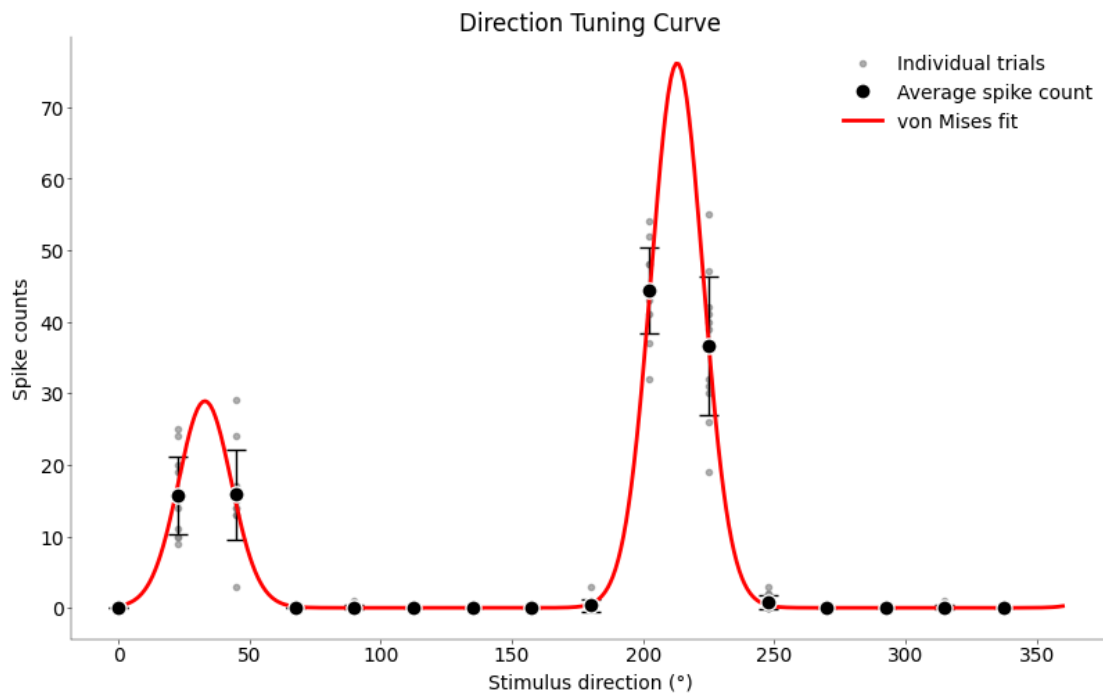
```

```

test_neurons = [28, 29, 37]
for idx, neuron in enumerate(test_neurons):
    print(f"\n--- Neuron {neuron} ---")
    dirs_sorted, counts_sorted = get_data(spikes, neuron)
    # The tuningCurve function handles the plotting if show=True
    p_opt = tuningCurve(counts_sorted, dirs_sorted, show=True)
    print(f"Fitted parameters [alpha, kappa, nu, phi]: {np.round(p_opt, 2)}")

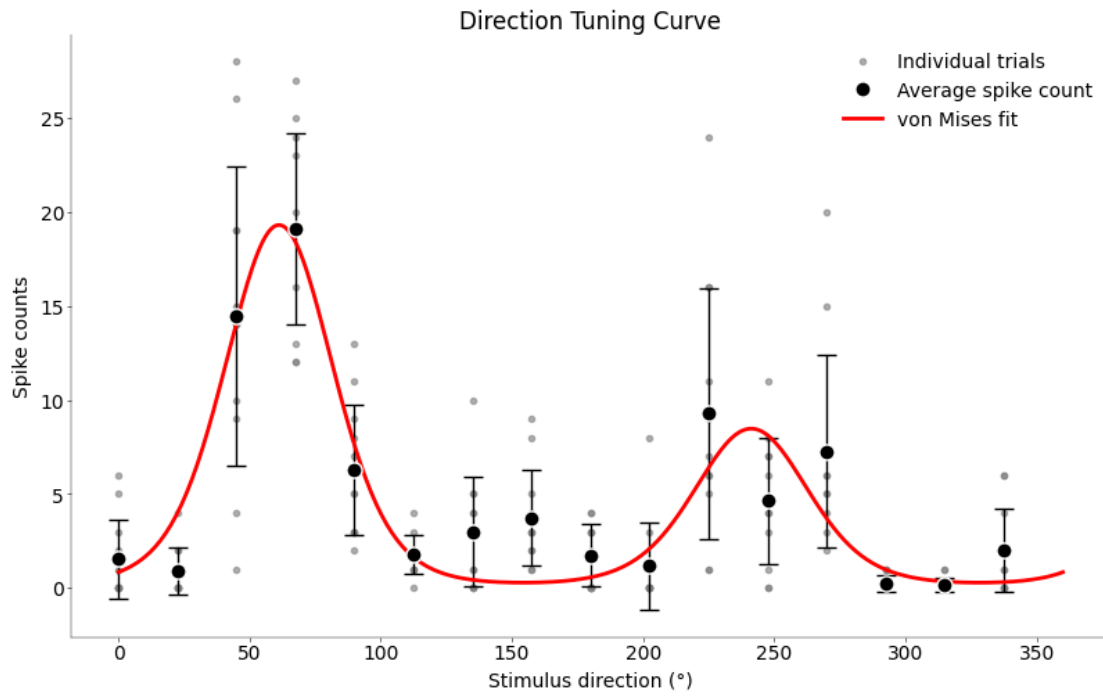
```

--- Neuron 28 ---



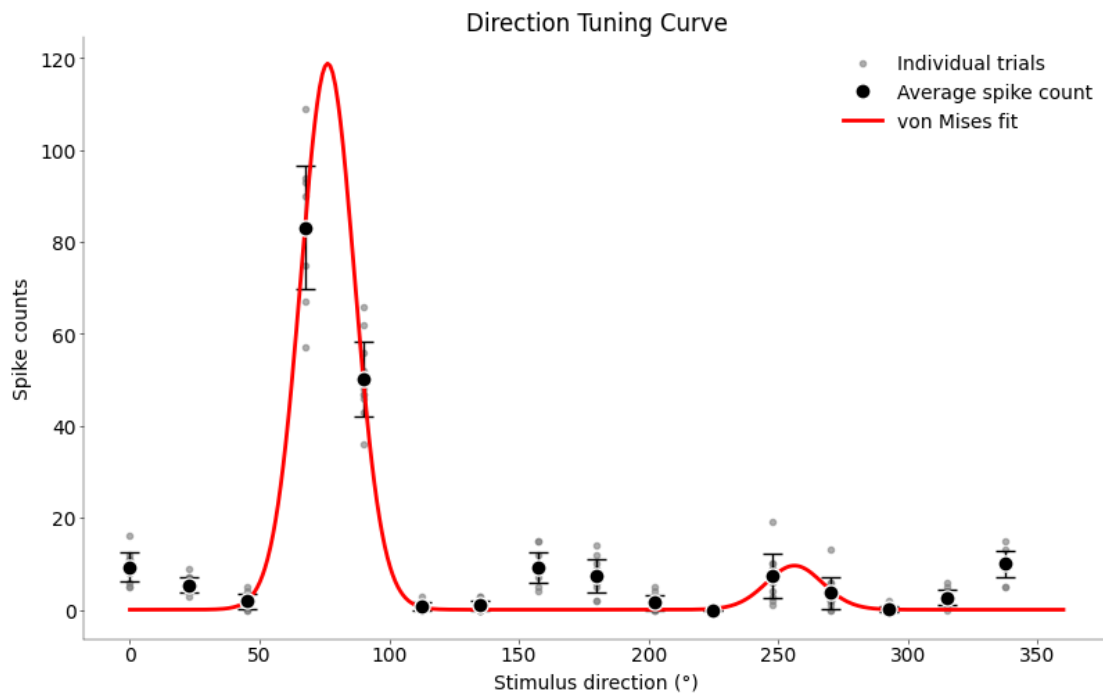
Fitted parameters [alpha, kappa, nu, phi]: [4.33 8.19 0.48 212.99]

--- Neuron 29 ---



Fitted parameters [alpha, kappa, nu, phi]: [2.96 1.9 0.41 61.23]

--- Neuron 37 ---



Fitted parameters [alpha, kappa, nu, phi]: [4.78 7.33 1.26 76.28]

1.5 Task 4: Permutation test for direction tuning

Implement a permutation test to quantitatively assess whether a neuron is direction/orientation selective. To do so, project the vector of average spike counts, $m_k = \frac{1}{N} \sum_j x_{jk}$ on a complex exponential with two cycles, $v_k = \exp(\psi i \theta_k)$, where θ_k is the k -th direction of motion in radians and $\psi \in 1, 2$ is the fourier component to test (1: direction, 2: orientation). Denote the projection by $q = m^T v$. The magnitude $|q|$ tells you how much power there is in the ψ -th fourier component.

Estimate the distribution of $|q|$ under the null hypothesis that the neuron fires randomly across directions by running 1000 iterations where you repeat the same calculation as above but on a random permutation of the trials (that is, randomly shuffle the entries in the spike count matrix x). The fraction of iterations for which you obtain a value more extreme than what you observed in the data is your p-value. Implement this procedure in the function `testTuning()`.

Illustrate the test procedure for one of the cells from above. Plot the sampling distribution of $|q|$ and indicate the value observed in the real data in your plot.

How many cells are tuned at $p < 0.01$?

Grading: 3 pts

```
[24]: def testTuning(
    counts: np.ndarray,
    dirs: np.ndarray,
    psi: int = 1,
    niters: int = 1000,
    show: bool = False,
    random_seed: int = 2046,
) -> Tuple[float, float, np.ndarray]:
    """Plot the data if show is True, otherwise just return the fit.

    Parameters
    -----

    counts: np.array, shape=(total_n_trials, )
        the spike count during the stimulation period

    dirs: np.array, shape=(total_n_trials, )
        the stimulus direction in degrees

    psi: int
        fourier component to test (1 = direction, 2 = orientation)

    niters: int
        Number of iterations / permutation
```

```

show: bool
    Plot or not.

random_seed: int
    Random seed for reproducibility.

Returns
-----
p: float
    p-value
q: float
    magnitude of Fourier component

qdistr: np.array
    sampling distribution of |q| under the null hypothesis

"""

# -----
# calculate m, nu and q (0.5 pts)
# -----

unique_dirs = np.unique(dirs)
m = np.array([counts[dirs == dir].mean() for dir in unique_dirs])
nu = np.exp(psi * 1j * np.deg2rad(unique_dirs))
q = np.abs(m.T @ nu)

# -----
# Estimate the distribution of q under the H0 and obtain the p value (1 pt)
# -----
# ensure reproducibility using a random number generator
# hint: access random functions of this generator
rng = np.random.default_rng(random_seed)

qdistr = []
counts_rag = counts.copy()
from tqdm import tqdm

for _ in range(niters):
    rng.shuffle(counts_rag)
    m0 = np.array([counts_rag[dirs == dir].mean() for dir in unique_dirs])
    q0 = np.abs(m0.T @ nu)
    qdistr.append(q0)

qdistr = np.array(qdistr)
p = np.mean(qdistr > q)

```

```

if show:
    # add plotting code here
    plt.figure(figsize=(5, 3))

    plt.hist(
        qdistr,
        bins=50,
        weights=np.ones_like(qdistr) / len(qdistr),
    )
    plt.axvline(q, linestyle="--", color="red", label=f"real  $|q|$  \n(p={p:.
↪3f})")
    plt.ylabel("Fraction of runs")
    plt.xlabel(" $|q|$ ")
    plt.title(f"Null distribution ({psi}-order)")
    plt.legend()
    plt.plot()

return p, q, qdistr

```

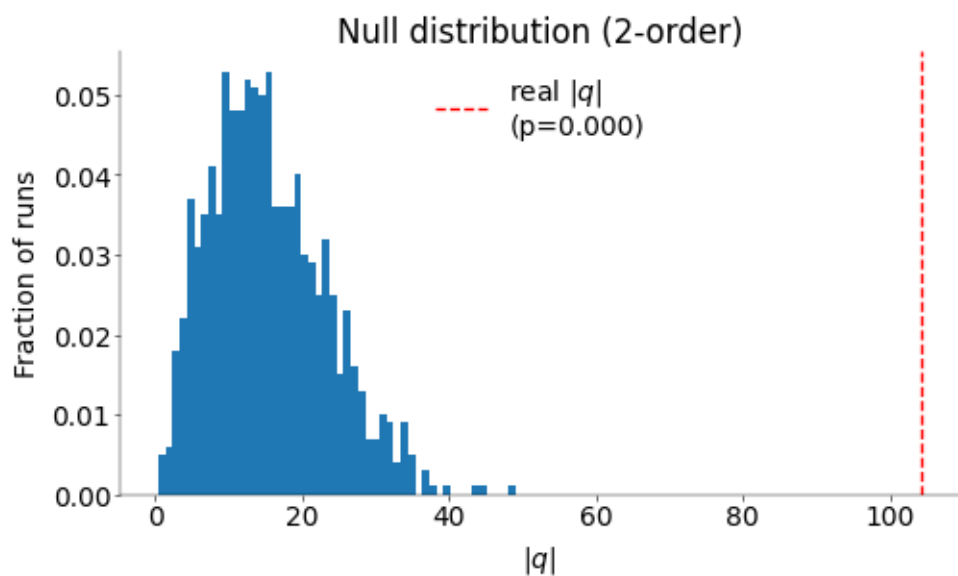
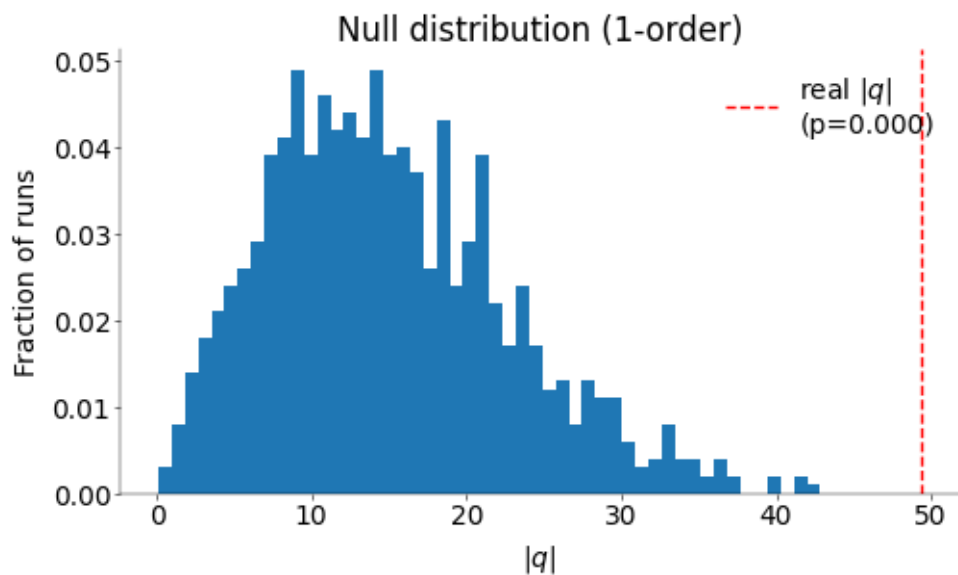
Show null distribution for the example cell:

```

[25]: # -----
# Plot null distributions for example cells 28 & 29. (1 pt)
# -----
neuron = 28
print(f"\n--- Neuron {neuron} ---")
dirs_sorted, counts_sorted = get_data(spikes, neuron)
# The tuningCurve function handles the plotting if show=True
p_1, q_1, _ = testTuning(counts_sorted, dirs_sorted, psi=1, show=True)
p_2, q_2, _ = testTuning(counts_sorted, dirs_sorted, psi=2, show=True)

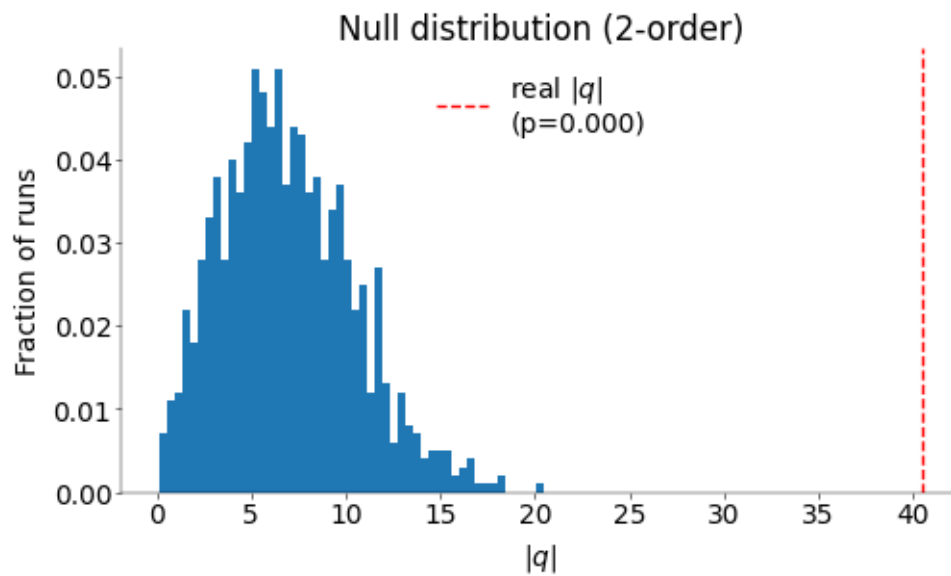
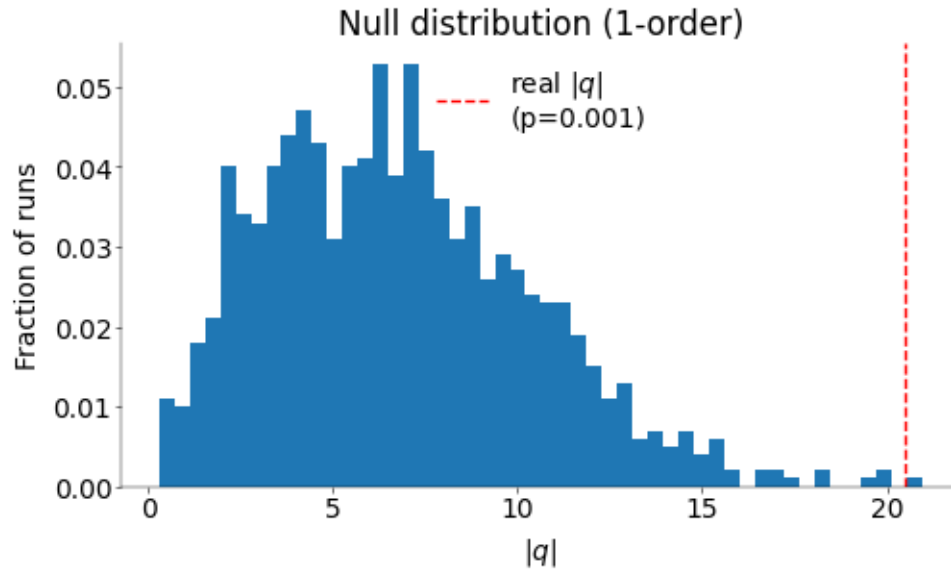
```

--- Neuron 28 ---



```
[26]: neuron = 29
print(f"\n--- Neuron {neuron} ---")
dirs_sorted, counts_sorted = get_data(spikes, neuron)
# The tuningCurve function handles the plotting if show=True
p_1, q_1, _ = testTuning(counts_sorted, dirs_sorted, psi=1, show=True)
p_2, q_2, _ = testTuning(counts_sorted, dirs_sorted, psi=2, show=True)
```


--- Neuron 29 ---



Test all cells for orientation and direction tuning

```
[27]: # -----  
# Test all cells for orientation / direction tuning.  
# Which ones are selective? (0.5 pts)  
# -----
```

```

unique_neurons = np.unique(spikes["Neuron"])
p_1_list = []
p_2_list = []

for neuron in unique_neurons:
    dirs_sorted, counts_sorted = get_data(spikes, neuron)
    p_1_list.append(testTuning(counts_sorted, dirs_sorted, psi=1,
↪show=False)[0])
    p_2_list.append(testTuning(counts_sorted, dirs_sorted, psi=2,
↪show=False)[0])

p_1_list = np.array(p_1_list)
p_2_list = np.array(p_2_list)

direction_tuned_neurons = unique_neurons[p_1_list < 0.01]
orientation_tuned_neurons = unique_neurons[p_2_list < 0.01]

```

Number of direction tuned neurons:

```
[28]: print("Number of direction tuned neurons: ", len(direction_tuned_neurons))
      print(direction_tuned_neurons)
```

Number of direction tuned neurons: 12
[13 20 24 25 27 28 29 31 32 37 38 40]

Number of orientation tuned neurons:

```
[29]: print("Number of orientation tuned neurons: ", len(orientation_tuned_neurons))
      print(orientation_tuned_neurons)
```

Number of orientation tuned neurons: 34
[2 3 6 7 8 10 12 13 14 15 16 17 18 20 21 22 23 24 25 26 27 28 29 30
31 32 33 34 36 37 38 39 40 41]